eBL-ATF Specification

▼ Contents

Lines 3
@-lines 4
$-lines 5
Note lines 6
Parallel lines 7
Translation lines 7
Text lines 8
Text 8
- Shifts 8
- Presence 9
- Columns 11
- Non-normalized text 12
  - Glosses 14
  - Commentary protocols 14
  - Word 15
- Normalized Akkadian 19
- Greek 20

Chapters 20
Validation 22
Labels 23

eBL-ATF is based on Oracc-ATF
but is not fully compatible with other ATF flavours. eBL-ATF uses UTF-8 encoding. The grammar definitions below use EBNF
(ISO/IEC 14977 : 1996(E)).

The EBNF grammar below is an idealized representation of the eBL-ATF as it does not deal with ambiguities and implementation details necessary to create the domain model in practice. A fully functional grammar is defined in
ebl-atf.lark.
The file uses the EBNF variant of the Lark parsing library.
See Grammar Reference
and Lark Cheat Sheet.

eBL-ATF can be empty or consist of lines separated by a newline character.

```
ebl-atf = [ line, { eol, line } ];

free-text = { any-character }-;
word-character = ? A-Za-z ?;
lower-case-letter = ? a-z ?;
any-character = ? any UTF-8 character ?;
decimal-digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';
number = { decimal-digit }-;

eol = ? end of line ?;
```

Common grammar used in multiple places.

```
seal = 'seal', number;
surface = 'obverse' | 'reverse' | 'left' | 'right' | 'top' | 'bottom'
        | face | generic-surface | edge;
face = 'face', ' ', lower-case-letter;
edge = 'edge', ' ', lower-case-letter;
generic-surface = 'surface', ' ', free-text;

object = 'tablet' | 'envelope' | 'prism' | 'bulla' | fragment | generic-object;
fragment = 'fragment', ' ', free-text;
generic-object = 'object', ' ', free-text;

status = "'" | '?' | '!' | '*';

markup = { markup-part }-;
markup-part = emphasis
            | akkadian
            | sumerian
            | emesal
            | markup-text
            | bibliography;
markup_token_part: "@" MARKUP_TOKEN "{" note_text "}"
MARKUP_TOKEN: "i" | "sup" | "sub" | "b"
akkadian = '@akk{', non-normalized-text, '}'; (* Default language is %akk *)
sumerian = '@sux{', non-normalized-text, '}'; (* Default language is %sux *)
emesal = '@es{', non-normalized-text, '}'; (* Default language is %es *)
bibliography = '@bib{', escaped-text, ( '@', escaped-text,) '}';
url = '@url{', url, '}', ('{', optional-display-text, '}');
escaped-text = { ( markup-character - '\' ) | '\@' | '\{' | '\}' | '\\' };
markup-text = { markup-character };
markup-character = any-character - ( '@' | '{' | '}' );
```

Lines

A line can be either *empty*, *control* or *text line*. Text lines contain transliterated text. Other lines do not currently have special semantics. Continuation lines (starting with space) are not supported.

```
line = empty-line
     | dollar-line
     | at-line
     | note-line
     | text-line
     | parallel-line
     | translation-line

     | control-line;

empty-line = '';

control-line = '=:' | '&', { any-character };
```

@-lines

@-lines are used for structural tags. Several kinds of structure may be indicated using this mechanism: physical structure, e.g., objects, surfaces; manuscript structure, i.e., columns; and document structure, e.g., divisions and colophons.

```
at-line = seal | column | heading | discourse | object-with-status | surface-wit
        | divisions | composite;

surface-with-status = surface, [ ' ' ], { status };

object-with-status = object, [ ' ' ], { status };

column = 'column ', number, [ ' ' ], { status };

heading = 'h', number, [ ' ', markup ];

discourse = 'catchline' | 'colophon' | 'date' | 'signature' | 'signatures'
          | 'summary'  | 'witnesses';

divisions = 'm=division ', free-text, [ ' ', number ];

composite = composite_composite | composite_start | composite_end | composite_mi
composite_start = 'div ', free-text, [ ' ', number ];
composite_end = 'end ', free-text;
composite_composite: 'composite';
composite_milestone: 'm=locator ', free-text, [ ' ', number ];
```

$-lines

$-lines are used to indicate information about the state of the text or object, or to describe features on the object which are not part of the transliteration proper.

- State: `$ <qualification> <extent> <scope> <state> <status>`, e.g.: `$ 3 lines blank` or `$ rest of obverse missing`.
- Loose: `$ (<free text>)`, e.g.: `$ (head of statue broken)`
- Ruling: `$ (single | double | triple) ruling`, e.g.: `$ double ruling`.
- Image: `$ (image N = <free text>)`, e.g.: `$ (image 1 = numbered diagram of triangle)`

```
dollar-line = '$', [ ' ' ], ( strict | '(', strict, ')' | loose );
strict = state | ruling | image | seal

state = [ qualification ], [' ', extend ], [' ', scope ], [ ' ', state-name ],
        [ ' ', dollar-status ]; (* At least one column is required. *)

qualification = 'at least' | 'at most' | 'about';

extent = 'several' | 'some' | number | range | 'rest of' | 'start of'
       | beginning of | 'middle of' | 'end of';

scope = object | surface | 'column' | 'columns' | 'line' | 'lines' | 'case'
      | 'cases' | 'side' | 'excerpt' | 'surface';

state-name = 'blank' | 'broken' | 'effaced' | 'illegible' | 'missing'
           | 'traces' | 'omitted' | 'continues';

range = number, '-', number;

loose = '(', free-text, ')';

ruling = ('single' | 'double' | 'triple'), ' ', 'ruling',
         [ [ ' ' ], dollar-status];

image = '(image ' number, [ lower-case-letter ], ' = ', free-text, ')';

dollar-status = '*' | '?' | '!' | '!?' | '°';
```

See: ATF Structure Tutorial

Note lines

```
note-line = '#note: ', markup;
```

Parallel lines

```
parallel-line = '// ', [ 'cf. ' ],
                ( parallel-composition | parallel-text | parallel-fragment );

parallel-composition = '(',  { any-character }-, ' ', line-number,  ')';

parallel-text = genre, ' ', category, '.', index, ' ',
                [ stage, ' ',  [ version, ' ' ], chapter , ' ' ], line-number;
genre = 'L' | 'D' | 'Lex' | 'Med' | 'Mag' | 'Šui' | 'Sum'
category = { 'I' | 'V' | 'X' | 'L' | 'C' | 'D' | 'M' }-;
           (* Must be a valid numeral. *)
stage = 'Ur3'  | 'OA'  | 'OB' | 'OElam' | 'PElam'  | 'MB' |
        'MElam'| 'MA'  | 'Hit' | 'NA' | 'NB' | 'NElam'  | 'LB' |
        'Per'  | 'Hel' | 'Par' | 'Uruk4' | 'JN' | 'ED1_2' |
        'Fara' | 'PSarg' | 'Sarg' | 'Aram' | 'Luw' | 'Unc' | 'SB';
version  = '"', { any-character }-, '"';
chapter =  '"', { any-character }-, '"';

parallel-fragment = 'F ', museum-number, ' ', [ '&d ' ],
                    [ object-label, ' ' ],
                    [ surface-label, ' ' ],
                    [ column-label, ' ' ],
                    line-number;
museum-number = ? .+?\.[^.]+(\.[^.]+)? ?;
```

Translation lines

```
translation-line = '#tr', [ '.', language-code ],
                   [ '.', translation-extent ], ': ', markup;
                   (* If omitted the language-code is 'en'. *)
language-code = ? ISO 639-1 language code ?;
translation-extent = '(', [ label, ' ' ] , line-number, ')';
```

See:

- ISO 639-1
- Translations

Text lines

```
text-line = line-number, '. ', text;

line-number = line-number-range | single-line-number;
line-number-range = single-line-number, '-', single-line-number;
single-line-number = [ word-character, '+' ], { decimal-digit }-, [ prime ],
                     [ word-character ];
prime = "'" | '′' | '’';
```

See: ATF Inline Tutorial
and ATF Quick Reference

Text

```
text = ( non-normalized-text | shifted-text ), { word-separator, shifted-text };
shifted-text = non-normalized-shift, word-separator, non-normalized-text
             | normalized-akkadian-shift, word-separator, normalized-akkadian;
word-separator = ' ';
```

Shifts

Shifts change the language and normalization of the subsequent words until another shift or the end of the line. Shifts are marked with `%` followed by a language code. If no shifts are present *Akkadian* is used as the default language.

Shift	Language	Variety	Normalized	Parsed to standard signs
`%n`	Akkadian		Yes	No
`%ma`	Akkadian	Middle Assyrian	No	Yes
`%mb`	Akkadian	Middle Babylonian	No	Yes
`%na`	Akkadian	Neo-Assyrian	No	Yes
`%nb`	Akkadian	Neo-Babylonian	No	Yes
`%lb`	Akkadian	Late Babylonian	No	Yes
`%sb`	Akkadian	Standard Babylonian	No	Yes
`%a`	Akkadian		No	Yes
`%akk`	Akkadian		No	Yes
`%eakk`	Akkadian	Early Akkadian	No	Yes
`%oakk`	Akkadian	Old Akkadian	No	Yes
`%ur3akk`	Akkadian	Ur III Akkadian	No	Yes
`%oa`	Akkadian	Old Assyrian	No	Yes
`%ob`	Akkadian	Old Babylonian	No	Yes
`%sux`	Sumerian		No	Yes
`%es`	Sumerian	Emesal	No	Yes
`%e`	Sumerian	Emesal	No	Yes
`%grc`	Greek		No	No
`%akkgrc`	Akkadian	In Greek characters	No	No
`%suxgrc`	Sumerian	In Greek characters	No	No
`%hit`	Hittite		No	Yes

Any other shifts are considered valid and have language *Unknown*. *Akkadian* and *Unknown* are lemmatizable.

```
 non-normalized-shift: shift - normalized-akkadian-shift;
normalized-akkadian-shift = '%n';
shift = '%', { word-character }-;
```

Presence

A presence cannot be nested within itself.

Presence Type	Open	Close	Scope
Intentional Omission	`<(`	`)>`	Top-level, Word
Accidental Omission	`<`	`>`	Top-level, Word
Removal	`<<`	`>>`	Top-level, Word
Broken Away	`[`	`]`	Top-level, Word, Grapheme
Perhaps Broken Away	`(`	`)`	Top-level, Word

Constraints:

- *Intentional Omission*: Cannot be inside *Accidental Omission*.
- *Accidental Omission*: Cannot be inside *Intentional Omission*.
- *Broken Away*: Cannot be inside *Perhaps Broken Away* (E.g. `(x) [(x)] (x)` not `(x [x] x)`).
- *Perhaps Broken Away*: Can be inside of *Broken Away*, and must be fully in or out (E.g. `[(x)] (x)` not `[(x] x)`). Cannot be inside *Accidental Omission* or *Intentional Omission*.

See: ATF Inline Tutorial

```
open_presence = { open-broken-away
                | open-perhaps-away
                | open-intentional-omission
                | open-accidental-omission
                | open-removal }+;
close_presence = { close-broken-away
                 | close-perhaps-away
                 | close-intentional-omission
                 | close-accidental-omission
                 | close-removal }+;

open-intentional-omission = '<(';
close-intentional-omission = ')>';

accidental-omission = open-accidental-omission | close-accidental-omission;
open-accidental-omission = '<';
close-accidental-omission = '>';

open-removal = '<<';
close-removal = '>>';

broken-away = open-broken-away-open | close-broken-away;
open-broken-away = '[';
close-broken-away = ']';

perhaps-broken-away = open-perhaps-broken-away | close-perhaps-broken-away;
open-perhaps-broken-away = '(';
close-perhaps-broken-away = ')';
```

Columns

Single `&` is a column separator. `&` followed by a number means that the following column spans the number of columns. If the first column spans multiple columns `&`+number can be put in the beginning of the line. If `&` is at the beginning the first column will be empty. Columns are not lemmatizable or alignable.

```
column = '&', { decimal-digit };
```

Non-normalized text

Text is a series of tokens separated by a word separator (space). Sometimes the separator is ignored (see Word below) or can be omitted.

Token Type	**Definition**	**Lemmatizable**	**Alignable**
WordOmitted	`ø`	No	No
Tabulation	`($___$)`	No	No
Divider	`:'`, `:"`, `:.`, `::`, `:?`, `:`, `;`, or `/`	No	No
Egyptian Metrical Feet Separator	`•`	No	No
Line Break	`\\|`	No	No
Commentary Protocol	`!qt`, `!bs`, `!cm`, or `!zz`	No	No
Erasure	`°` + erased + `\` + overwritten + `°`	Special	Special
Word	Readings or graphemes separated by a joiner.	Maybe	Maybe
Lone Determinative	A word consisting only a determinative part.	No	No
Document Oriented Gloss	`{(` or `)}`	No	No
Presence	Enclosure markers (see *Presence* section)	No	No

Notes:

- *Divider*: Must be followed by the word separator or end of line. Can be followed by flags and modifiers and surrounded with broken away.
- *Egyptian Metrical Feet Separator*: Can be within a word or standing alone between words. Can be followed by flags and surrounded with broken away and presence indicators.
- *Line Break*: Must be followed by the word separator or end of line. Can be followed by flags and modifiers and surrounded with broken away.
- *Erasure*: Must be followed by a separator or end of line. Erasure markers and erased words are not lemmatizable or alignable, but words written over erasure can be.
- *Word*: See Word below for full definition.
- *Lone Determinative*: See Word and Glosses below.
- *Commentary Protocol*: See Commentary Protocols below.

```
non-normalized-text = token, { [ word-separator ], token };
       (* Word separator can be omitted after an opening bracket or before
          a closing bracket. Commentary protocols and dividers must be
          surrounded by word separators. *)

token = commentary-protocol
      | divider
      | divider-variant
      | egyptian-metrical-feet-separator
      | line-break
      | tabulation
      | column
      | erasure
      | word
      | determinative
      | close-broken-away
      | close-perhaps-broken-away
      | close-intentional-omission
      | close-accidental-omission
      | close-removal
      | close-document-orionted-gloss
      | open-broken-away
      | open-perhaps-broken-away
      | open-intentional-omission
      | open-accidental-omission
      | open-removal
      | open-document-oriented-gloss;

word-ommited = 'ø';
tabulation = '($___$)';

divider-variant = ( variant-part | divider ), variant-separator,
                  ( variant-part | divider );
divider = divider-symbol, modifier, flag;
divider-symbol = ":'" | ':"' | ':.' | '::' | ':?' | ':' | ';' | '/';

egyptian-metrical-feet-separator = "•", flag;

line-break: '|';
```

```
commentary-protocol = '!qt' | '!bs' | '!cm' | '!zz';

document-oriented-gloss = '{(', token, { [word-separator], token } ,')}';

shift = '%', { word-character }-;

erasure = '°', [ erasure-part ] '\', [ erasure-part ], '°';
erasure-part = ( divider | word | lone-determinative ),
               { word-separator, ( divider | word | lone-determinative ) };
```

Glosses

Glosses cannot be nested within other glosses in the same scope.

Gloss Type	Open	Close	Scope	Constraints
Document Oriented Gloss	`{(`	`)}`	Top-level	
Linguistic Gloss	`{{`	`}}`	Word	
Determinative	`{`	`}`	Word	
Phonetic Gloss	`{+`	`}`	Word	Cannot appear alone.

Examples:

- *Document Oriented Gloss*: `{(1(u))}` `{(%a he-pi₂ eš-šu₂)}`
- *Linguistic Gloss*: `du₃-am₃{{mu-un-<(du₃)>}}`
- *Determinative*: `{d}utu` `larsa{ki}`
- *Phonetic Gloss*: `{+u₃-mu₂}u₂-mu₁₁` `AN{+e}`

See: ATF Inline Tutorial

Commentary protocols

The Commentary protocol is used to change the display color of the text that follows it. It has to be declared at the beginning of every line. Once declared, it is valid until another protocol replaces it.

Protocol	Description
`!qt`	`Quotation`
`!bs`	`Base Text`
`!cm`	`Commentary`
`!zz`	`Uncertain`

See: Base Text and Commentary

Word

A *lone determinative* is a special case of a word consisting only a single determinative. A word is lemmatizable and alignable if:

- It is not erased.
- It is not lone determinative.
- It does not contain variants.
- It does not contain unclear signs.
- It does not contain unidentified signs.
- It does not contain unknown number of signs.
- The language is lemmatizable.
- The language is not normalized.

A word cannot start or end with a joiner and word separators adjecent to a joiner are ignored. E.g. `... -a]d` is equivalent to `...-a]d`.

```
word = [ open-any ],
       ( inline-erasure | parts ), { part-joiner, ( inline-erasure | parts ) },
       [ close-any ];

inline-erasure = '°', [ parts ], '\', [ parts ], '°';

parts = part, { [ part-joiner ], part };
part = value
     | determinative
     | linguistic-gloss
     | phonetic-gloss;

linguistic-gloss = '{{', gloss-body, '}}';
phonetic-gloss = '{+', gloss-body, '}';
determinative = '{', gloss-body, '}';
gloss-body = { open-intentional-omission | open-accidental-omission
             | open-removal },
             value, { part-joiner, value },
             { close-intentional-omission | close-accidental-omission
             | close-removal };

part-joiner = [ inword-newline ], [ close_presence ], [ joiner ],
              [ open_presence ];
              (* The joiner can be omitted next to determinative,
                 phonetic-gloss, or linguistic gloss. *)

joiner = { word-separator } ( '-' | '+' | '.' | ':' | ',' ) { word-separator };
inword-newline = ';';

value = unknown
      | value-with-sign
      | reading
      | compound-grapheme
      | logogram
      | surrogate
      | number
      | variant;

variant = variant-part, { variant-separator , variant-part };
```

```
variant-part = unknown
             | unknown-number-of-signs
             | value-with-sign
             | reading
             | compound-grapheme
             | logogram
             | surrogate
             | number;

number = decimal-digit, { [ invalue-broken-away ], decimal-digit },
         modifier, flag;

surrogate = logogram, ['<(', value, { '-', value } ,')>'];
logogram = logogram-character, { [ invalue-broken-away ], logogram-character },
           [ sub-index ], modifier, flag;
logogram-character = 'A' | 'Ā' | 'Â' | 'B' | 'D' | 'E' | 'Ē' | 'Ê' | 'G' | 'H'
                   | 'I' | 'Ī' | 'Î' | 'Y' | 'K' | 'L' | 'M' | 'N' | 'P' | 'Q'
                   | 'R' | 'S' | 'Ṣ' | 'Š' | 'T' | 'Ṭ' | 'U' | 'Ū' | 'Û' | 'W'
                   | 'Z' | 'Ḫ' | 'ʾ';

value-with-sign = ( reading | logogram | number ),
                 '(', ( compound-grapheme | grapheme ), ')';
reading = reading-character, { [ invalue-broken-away ], reading-character },
        [ sub-index ], modifier, flag;
reading-character = 'a' | 'ā' | 'â' | 'b' | 'd' | 'e' | 'ē' | 'ê' | 'g' | 'h'
                | 'i' | 'ī' | 'î' | 'y' | 'k' | 'l' | 'm' | 'n' | 'p' | 'q'
                | 'r' | 's' | 'ṣ' | 'š' | 't' | 'ṭ' | 'u' | 'ū' | 'û' | 'w'
                | 'z' | 'ḫ' | 'ʾ' | decimal-digit;
sub-index = { sub-index-character }-;

compound-grapheme = '|', compound-part, { compound-operator, compound-part },
                    '|';
compound-part = '(' grapheme-variant { compound-operator, grapheme-variant } ')'
              | grapheme-variant
grapheme-variant = grapheme, { variant-separator, grapheme };
compound-operator = '.' | '×' | '%' | '&' | '+';

grapheme = grapheme-character,
           { [ invalue-broken-away ], grapheme-character },
           modifier,
           flag;
grapheme-character = word-character
                   | 'Ṣ' | 'Š' | 'Ṭ'
                   | 'ṣ' | 'š' | 'ṭ'
                   | decimal-digit
                   | sub-index-character - 'ₓ';

unknown-number-of-signs = '...';
unknown = ('X' | 'x'), flag;

invalue-broken-away: open-broken-away | close-broken-away;

variant-separator = '/';

flag = { damage | uncertain | correction | collation | no-longer-visible };
damage = '#';
uncertain = '?';
correction = '!';
collation = '*';
no-longer-visible = '°';

modifier = { '@', ( 'aš' | 'c' | 'f' | 'g' | 's' | 't' | 'n'
                  | 'z' | 'k' | 'r' | 'h' | 'v' | { decimal-digit }- ) };

sub-index-character = '₀' | '₁' | '₂' | '₃' | '₄' | '₅'
                    | '₆' | '₇' | '₈' | '₉' | 'ₓ';
```

Normalized Akkadian

```
normalized-akkadian = normalized-text, { break, normalized-text };
normalized-text = ( normalized-word | lacuna ),
                  { word-separator, ( normalized-word | lacuna ) };

break =  word-separator, ( caesura | foot-separator )  word-separator;
caesura = '||' | '(||)';
foot-separator = '|' | '(|)';

lacuna = { enclosure-open }, ellipsis, { enclosure-close };

normalized-word = { enclosure-open },
                [ ellipsis, [ between-strings ] ],
                akkadian-string,
                { between-strings, ( akkadian-string | ellipsis )
                | ellipsis, [ between-strings ], akkadian-string
                | akkadian-string
                },
                [ [ between-strings ], ellipsis ],
                [ normalization-modifier ],
                [ normalization-modifier ],
                [ normalization-modifier ],
                { enclosure-close };
normalization-modifier = damage | uncertain | correction;

between-strings = { enclosure }-, separator
                | separator, { enclosure }-
                | { enclosure }-
                | separator;
separator = '-';

ellipsis = '...';

enclosure = enclosure-open | enclosure-close;
enclosure-open = open-broken-away | open-perhaps-broken-away | open-emendation;
enclosure-close = open-broken-away | close-perhaps-broken-away | close-emendatio
open-emendation = '<';
close-emendation = '>';
```

```
akkadian-string = { akkadian-alphabet }-;
akkadian-alphabet = 'ʾ' | 'A' | 'B' | 'D' | 'E' | 'G' | 'H' | 'I' | 'K' | 'L'
                  | 'M' | 'N' | 'P' | 'S' | 'T' | 'U' | 'Y' | 'Z' | 'a' | 'b'
                  | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i' | 'k' | 'l' | 'm'
                  | 'n' | 'p' | 'q' | 'r' | 's' | 't' | 'u' | 'w' | 'y' | 'z'
                  | 'É' | 'â' | 'ê' | 'î' | 'û' | 'ā' | 'Ē' | 'ē' | 'ī' | 'Š'
                  | 'š' | 'ū' | 'ṣ' | 'ṭ' | '₄';
```

Greek

```
 greek = greek-token, { word-separator, greek-token };
greek-token = greek-word | column;
greek-word = { greek-writing | greek-presence },
             greek-writing,
             { greek-writing |  greek-presence };
greek-presence = open_presence | close_presence;
greek-writing = greek-letter
              | unknown-number-of-signs
              | unknown;
greek-letter = greek-alphabet, flag;
greek-alphabet = 'Α' | 'α' | 'Β' | 'β' | 'Γ' | 'γ' | 'Δ' | 'δ' | 'Ε' | 'ε'
               | 'Ζ' | 'ζ' | 'Η' | 'η' | 'Θ' | 'θ' | 'Ι' | 'ι' | 'Κ' | 'ϰ'
               | 'Λ' | 'λ' | 'Μ' | 'μ' | 'Ν' | 'ν' | 'Ξ' | 'ξ' | 'Ο' | 'ο'
               | 'Π' | 'π' | 'Ρ' | 'ϱ' | 'Σ' | 'σ' | 'ς' | 'Τ' | 'τ' | 'Υ'
               | 'υ' | 'Φ' | 'ϕ' | 'Χ' | 'χ' | 'Ψ' | 'ψ' | 'Ω' | 'ω';
```

Chapters

See Editorial-conventions-(Corpus).

```
chapter = chapter-line, { eol, eol, chapter-line };

chapter-line = { translation-line, eol }, line-variant, { eol, line-variant };
line-variant = reconstruction, { eol, manuscript-line };
reconstruction = text-line, [ eol, note-line ], { eol, parallel-line };

manuscript-line = { white-space }, siglum, ' ' , label, [ text-line ],
                  paratext;
paratext = { eol, { white-space },  ( dollar-line | note-line ) };
white-space = ? space or tab ?;

siglum = [ provenance ], period, [ type ], [ free-text - ( white-space | eol ) ]
provenance = 'Assa'
           | 'Ašš'
           | 'Huz'
           | 'Kal'
           | 'Kho'
           | 'Nin'
           | 'Tar'
           | 'Baba'
           | 'Bab'
           | 'Bor'
           | 'Cut'
           | 'Dil'
           | 'Isn'
           | 'Kiš'
           | 'Lar'
           | 'Met'
           | 'Nēr'
           | 'Nip'
           | 'Sip'
           | 'Shi'
           | 'Šad'
           | 'Ur'
           | 'Urk'
           | 'Ala'
           | 'Ama'
           | 'Emr'
           | 'Hat'
           | 'Ham'
           | 'Mar'
           | 'Meg'
           | 'Sus'
           | 'Uga'
           | 'Unc';
period = 'Ur3'
       | 'OA'
       | 'OB'
       | 'MB'
       | 'MA'
       | 'Hit'
       | 'NA'
       | 'NB'
       | 'LB'
       | 'Per'
       | 'Hel'
       | 'Par'
       | 'Unc';
type = 'Sch'
     | 'Com'
     | 'Quo'
     | 'Var'
     | 'Ex'
     | 'Par'
     | 'Amu'
     | 'MultCol'
     | 'Coll'
     | 'StuTea'
     | 'SchLen'
     | 'Prism'
     | 'Unc';
```

Validation

The ATF should be parseable using the specification above. In addition, all readings and signs must be correct according to the eBL sign list. Sometimes when the validation or parsing logic is updated existing transliterations can become invalid. It should still be possible to load these transliterations, but saving them results in an error until the syntax is corrected.

In addition, the combination of object, surface, column and line number should be unique for each text line.

Labels

```
 label = [ surface-label, ' ' ],  [ column-label, ' ' ];

column-label = roman-numeral, { status };
roman-numeral = { 'i' | 'v' | 'x' | 'l' | 'c' | 'd' | 'm' }-;
                (* Must be a valid numeral. *)

surface-label = ( 'o' | 'r' | 'b.e.' | 'e.' | 'l.e.' | 'r.e.' | 't.e.' ),
                { status };

object-label = ( 'tablet' | 'envelope' | 'prism' | 'bulla') { status }
```

See: Labels
and ATF Structure Tutorial